



19

Deploying a Next.js App

After learning about advanced Next.js concepts, it's time to learn how to deploy a Next.js app. The easiest way to deploy Next.js apps is by using the Vercel platform, provided by the company that develops the Next.js framework. After learning how to deploy our app on the Vercel platform, we are going to learn how to create a custom deployment setup using Docker.

In this chapter, we are going to cover the following main topics:

- Deploying a Next.js app with Vercel
- Creating a custom deployment setup for Next.js apps

Technical requirements

Before we start, please install all requirements from [Chapter 1, Preparing for Full-Stack Development](#), and [Chapter 2, Getting to Know Node.js and MongoDB](#).

The versions listed in those chapters are the ones used in the book. While installing a newer version should not be an issue, please note that certain steps might work differently on a newer version. If you are having an issue with the code and steps provided in this book, please try using the versions mentioned in [Chapter 1](#) and 2.

You can find the code for this chapter on GitHub:

<https://github.com/PacktPublishing/Modern-Full-Stack-React-Projects/tree/main/ch19>.

The CiA video for this chapter can be found at:

<https://youtu.be/ERBFy5mHwek>.

Deploying a Next.js app with Vercel

We are going to start by deploying our app on Vercel, a platform where we can deploy our apps for free in a simple and convenient way. Follow these steps to get started deploying our Next.js app with Vercel:

1. Copy the existing **ch18** folder to a new **ch19** folder by running the following command:

```
$ cp -R ch18 ch19
```

2. Open the **ch19** folder in VS Code.
3. Install the Vercel CLI tool as a global package with the following command:

```
$ npm install -g vercel@33.5.3
```

4. Run the Vercel CLI:

```
$ vercel
```

5. You will be asked to log in to Vercel. Select either of the login methods and follow the steps provided by Vercel to log in.
6. After successfully logging in, you will be asked questions about the deployment of your project, confirm all of them with the default values provided by pressing *Enter/Return* until the Vercel CLI attempts to build your project.

```
➔ ~/D/F/ch19 ↵ main > vercel
Vercel CLI 33.5.3
> > No existing credentials found. Please log in:
? Log in to Vercel github
> Success! GitHub authentication complete for me@omnidan.net
? Set up and deploy "~/Development/Full-Stack-React-Projects/ch19"? [Y/n] y
? Which scope do you want to deploy to? omnidan
? Link to existing project? [y/N] n
? What's your project's name? ch19
? In which directory is your code located? ./
Local settings detected in vercel.json:
Auto-detected Project Settings (Next.js):
- Build Command: next build
- Development Command: next dev --port $PORT
- Install Command: `yarn install`, `pnpm install`, `npm install`, or `bun install`
- Output Directory: Next.js default
? Want to modify these settings? [y/N] n
🔗 Linked to omnidan/ch19 (created .vercel)
🔍 Inspect: https://vercel.com/omnidan/ch19/7PhHxuGSJog1MGxReTYY8kKZgtvj [2s]
✅ Production: https://ch19-kwlu28dbp-omnidan.vercel.app [2s]
```

Figure 19.1 – Attempting to deploy our app to Vercel

7. While the project is building, you can visit the URL provided in the CLI to see the current state of the build process (make sure you are logged in to Vercel in the same browser), as shown in the following screenshot:

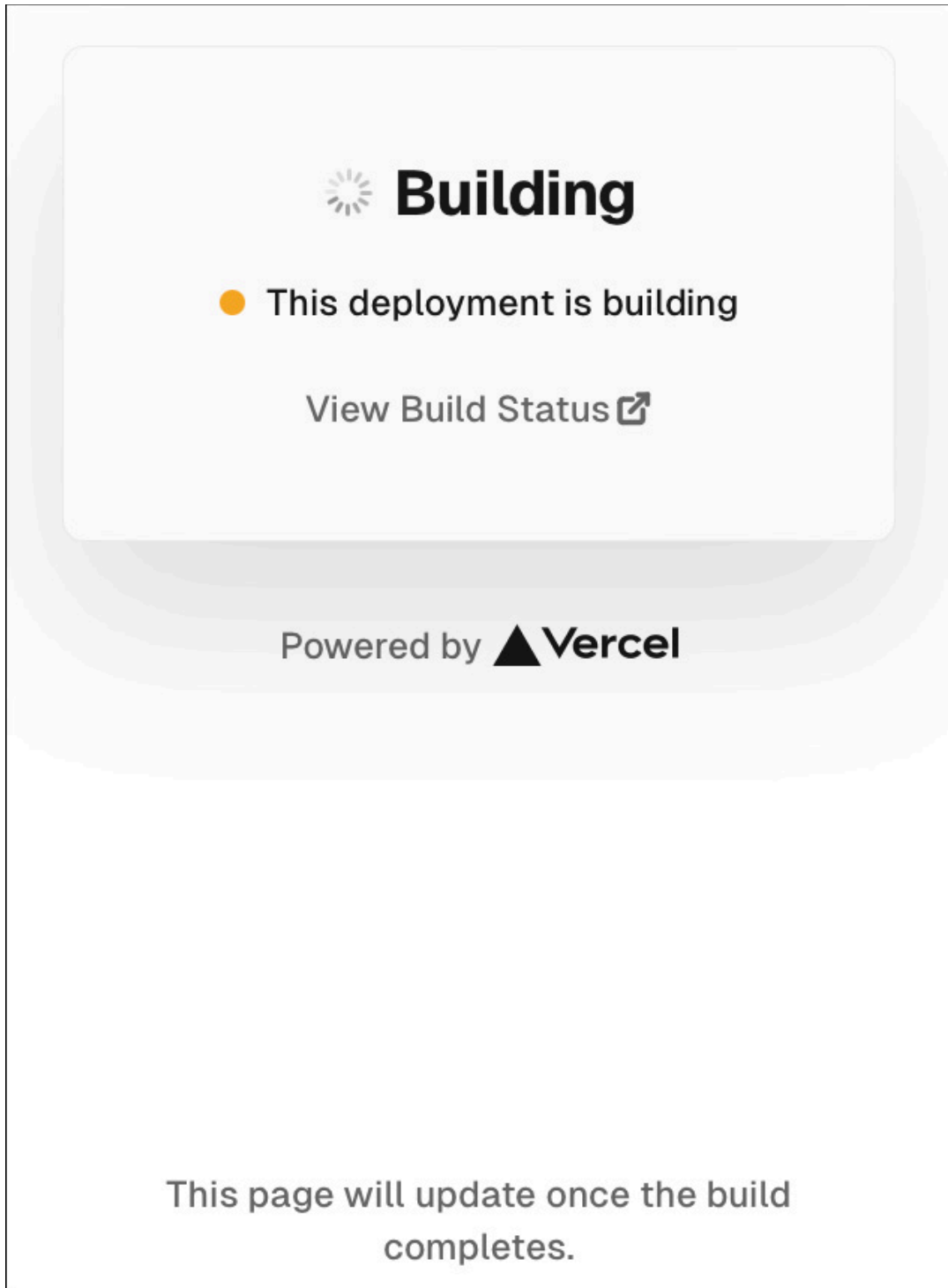


Figure 19.2 – Monitoring the build process in the browser

8. Unfortunately, the build fails because the `DATABASE_URL` environment variable is set to `mongodb://localhost:27017/blog`.

We now need to adjust this environment variable in Vercel.

Setting environment variables in Vercel

Follow these steps to set up the necessary environment variables in Vercel:

1. Re-use the existing database cluster created in MongoDB Atlas or follow the steps in the *Creating a MongoDB Atlas database* section of [Chapter 5](#) to create a new database cluster. You should now have a connection string for your database.
2. Verify that the connection string works by executing the following command:

```
$ mongosh "<connection-string>"
```

3. If you are re-using the existing database cluster, make sure to clear the database/collections, as the posts and users used to have a slightly different format in [Chapter 5](#)! Run the following commands inside the MongoDB Shell to clear the collections:

```
> db.posts.drop()  
> db.users.drop()
```

4. Go to <https://vercel.com/> and log in with the same login provider you used earlier.
5. You should see an overview of your projects, including the **ch19** project we created earlier via the Vercel CLI, as shown in the following screenshot:

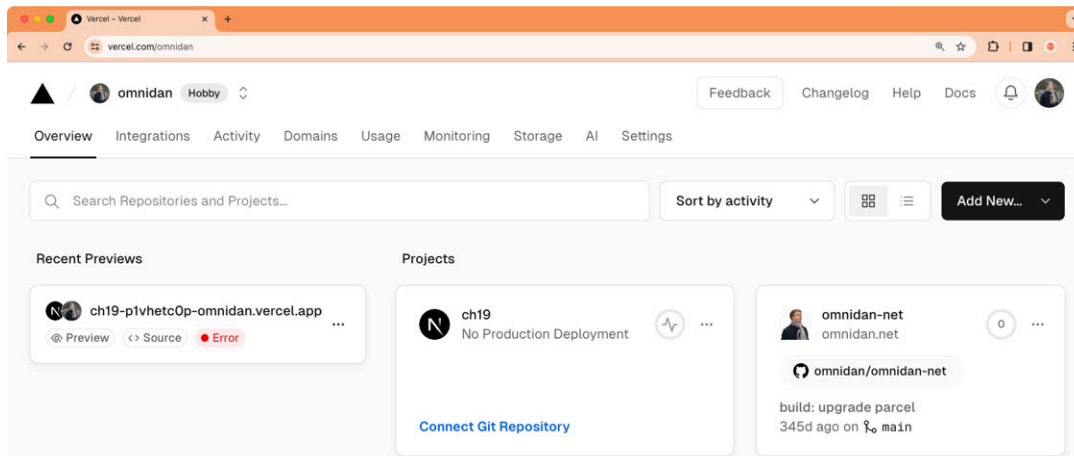


Figure 19.3 – The Vercel dashboard

6. Click on the **ch19** project, then go to the **Settings** tab, select **Environment Variables** on the sidebar, and create a new environment variable by entering **DATABASE_URL** as the **Key** and the previously obtained connection string as **Value**, as shown in the following screenshot:

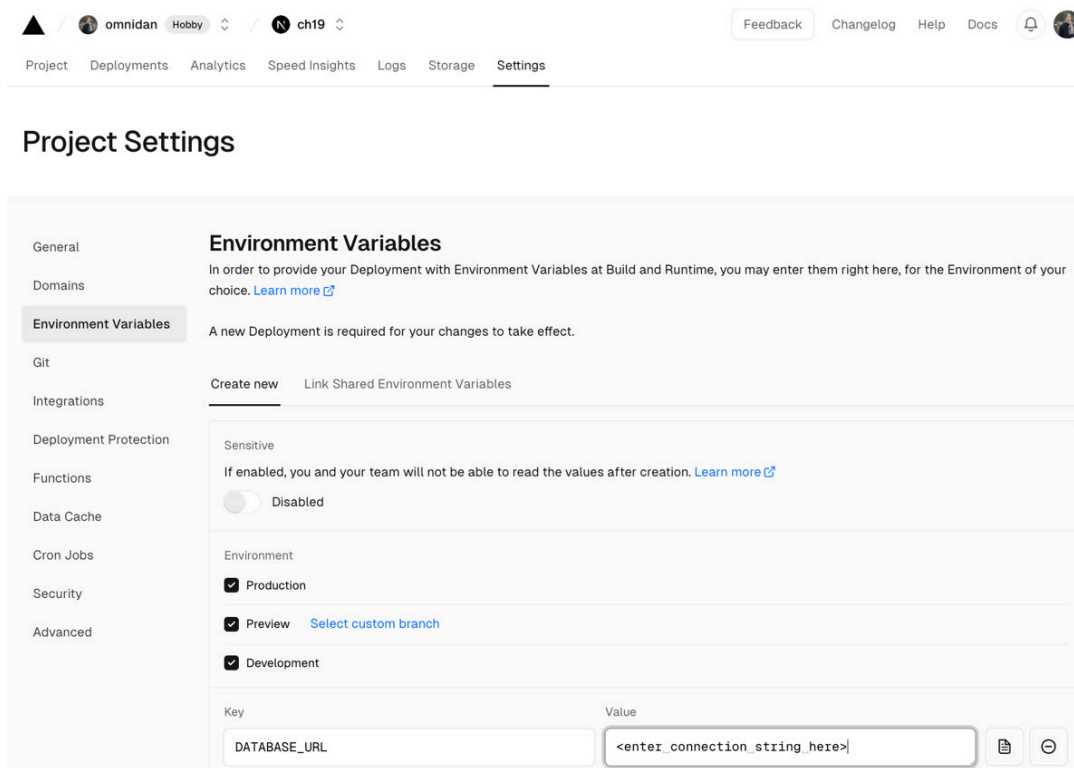


Figure 19.4 – Adding an environment variable in Vercel

NOTE

*For production apps, you would also set the **JWT_SECRET** environment variable to a random secret here. Additionally, you would set the **BASE_URL** environment variable to the URL of the production deployment of your app. For example, if the public URL of your blog is going to be **https://ch19-omnidan.vercel.app/**, you would set the **BASE_URL** to that.*

7. Click the **Save** button below the environment variables to save your changes.
8. Run the Vercel CLI again to attempt another deployment:

```
$ vercel
```

Alternatively, you can trigger a rebuild from the Vercel web UI.

9. You will see that it deploys successfully now, visit the **Preview** URL provided by the Vercel CLI in your browser to see our blog application loading successfully:

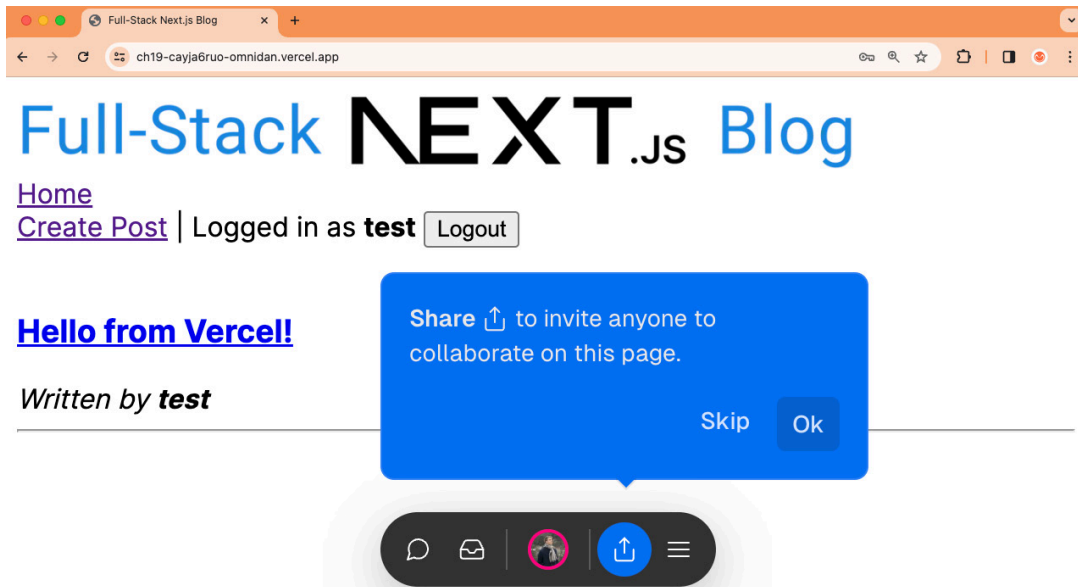


Figure 19.5 – Working “Preview” deployment of our app

Interestingly, Vercel CLI now made a **Preview** deployment for us. This is the default behavior in Vercel. It will first deploy to a **Preview** environment, where we can test everything out to make sure our app works fine. The **Preview** environment is only accessible if you are logged in via Vercel. We can also invite others to test our app here and add comments to it via the Vercel toolbar at the bottom.

10. Now that we have confirmed our app works, we can deploy it to production, as follows:

```
$ vercel --prod
```

The following screenshot shows a **Preview** and a **Production** deployment being made with the Vercel CLI:

```
➔ ~/D/F/ch19 ✎ main > vercel
Vercel CLI 33.5.3
🔍 Inspect: https://vercel.com/omnidan/ch19/2HXmvcW1cR6TmB4yurf8kr5LpAJq [3s]
✅ Preview: https://ch19-cayja6ruo-omnidan.vercel.app [3s]
📄 To deploy to production (ch19.vercel.app), run `vercel --prod`
➔ ~/D/F/ch19 ✎ main > vercel --prod
Vercel CLI 33.5.3
🔍 Inspect: https://vercel.com/omnidan/ch19/3kwA6ozErmsT15heLEDChQUG3Yy [1s]
✅ Production: https://ch19-f230clhd2-omnidan.vercel.app [1s]
```

Figure 19.6 – Deploying our app to “Preview” and “Production” environments with the Vercel CLI

Now our app is deployed on the **Production** environment and is accessible to anyone, without having to log in via Vercel!

NOTE

*The URL provided in the Vercel CLI output is not accessible to anyone; you need to use one of the domains specified in the **Domains** section on the Vercel dashboard. The default should be `https://ch19-<vercel-username>.vercel.app/`.*

As we can see, deploying our app with Vercel is very easy and convenient. However, in some cases, we want to deploy our app on our own infrastructure. Let’s learn how to create a custom deployment setup for Next.js apps now.

Creating a custom deployment setup for Next.js apps

We are now going to learn how to set up a custom deployment for Next.js apps using Docker. We have already learned the basics of deploying apps using Docker in [Chapter 5](#), so please refer to that chapter if anything is unclear, or if you need a refresher on Docker. Let’s get started setting up our Next.js app for a Docker deployment now:

1. First, we need to change the output format for Next.js to **standalone**.

This option tells Next.js to create a `.next/standalone` folder that only contains the necessary files for a production deployment, in-

cluding only the necessary **node_modules**. This folder can then be deployed without having to install **node_modules** again. Edit **next.config.mjs** and adjust the config, as follows:

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  output: 'standalone',
}
export default nextConfig
```

2. Now, we create a **.dockerignore** file to ignore certain files that should not be included in our image:

```
node_modules
.env*
.vscode
.git
```

3. Create a new **Dockerfile**, start by defining a **base** image from **node:20**:

```
FROM node:20 AS base
```

4. Then, define a new image for building the app, based on the **base** image:

```
FROM base AS build
```

5. Set the working directory to the **/app** folder and copy over the **package.json** and **package-lock.json** files:

```
WORKDIR /app
COPY package.json .
COPY package-lock.json .
```

6. Now, install all dependencies, and additionally install **sharp**, which is used by Next.js to resize and optimize images in production:

```
RUN npm install  
RUN npm install sharp
```

7. Copy over all files from our project:

```
COPY . .
```

8. Next, define the arguments for the build process. We are going to define all environment variables here because Next.js also uses them during the build process to statically build certain routes:

```
ARG DATABASE_URL  
ARG JWT_SECRET  
ARG BASE_URL
```

9. We can now run the **build** command, as follows:

```
RUN npm run build
```

10. Define a new image for the final app, based on the **base** image as well:

```
FROM base AS final
```

11. We also define the working directory:

```
WORKDIR /app
```

12. We set up the permissions to run our app as a special **nextjs** user instead of root:

```
RUN addgroup --system --gid 1001 nodejs  
RUN adduser --system --uid 1001 nextjs
```

13. Now, copy over the necessary files to run a standalone Next.js server from the **build** image:

```
COPY --from=build /app/public ./public
RUN mkdir -p .next
RUN chown nextjs:nodejs .next
COPY --from=build /app/.next/standalone ./
COPY --from=build /app/.next/static ./next/static
```

14. We define the **PORT**, **HOSTNAME**, and **NODE_ENV** variables:

```
EXPOSE 3000
ENV PORT 3000
ENV HOSTNAME "0.0.0.0"
ENV NODE_ENV production
```

15. Then, we execute the standalone Next.js server as the **nextjs** user we defined earlier:

```
USER nextjs
CMD ["node", "server.js"]
```

16. Make sure the database server is running in a Docker container.
17. Now we can build the Docker image by running the following command:

```
$ docker build \
  -t blog-nextjs \
  --build-arg "DATABASE_URL=mongodb://host.docker.internal:27017/blog" \
  --build-arg "JWT_SECRET=replace-with-random-secret" \
  --build-arg "BASE_URL=http://localhost:3000" \
  .
```

In the preceding command, we specified **blog-nextjs** as the name for our image and the necessary environment variables for building the image. Do not forget the dot (.) at the end of the command, as that is what specifies the build context, including where to look for the **Dockerfile**!

NOTE

You can check out the official example **Dockerfile** from Next.js for an up-to-date version:

<https://github.com/vercel/next.js/blob/canary/examples/with-docker/Dockerfile>

18. Finally, run a new Docker container, as follows:

```
$ docker run \
  -d \
  --name blog-app \
  -p 3000:3000 \
  -e "DATABASE_URL=mongodb://host.docker.internal:27017/blog" \
  -e "JWT_SECRET=replace-with-random-secret" \
  -e "BASE_URL=http://localhost:3000" \
  --restart unless-stopped \
  blog-nextjs
```

In the preceding command, we specified the running of a container with the name **blog-app** in the background (daemon mode) published to port **3000**, then specified the environment variables and told Docker to restart the container if it crashes. Lastly, we specified the image name, which is **blog-nextjs** (the image we built in the previous step).

19. Visit **http://localhost:3000** and you will see the blog running successfully!

Now that we have a Docker container, we could deploy it to a cloud service (or our own server), just like we did in [Chapter 5](#). While it is slightly more effort to set up a custom deployment for a Next.js app, it is still quite straightforward to do a simple setup!

For more advanced setups, such as multiple instances, you would need to set up a shared volume between the instances so the cache and optimized images can be shared (on Vercel, this is done automatically behind the scenes). However, such a setup is out of the scope of this book. You can check out the Next.js docs on self-hosting for more information

on how to do this: <https://nextjs.org/docs/app/building-your-application/deploying#self-hosting>.

Summary

In this chapter, we first learned how to deploy a Next.js app using Vercel. Then, we learned how to create a custom deployment setup using Docker.

In the next and final chapter, [***Chapter 20***](#), *Diving Deeper into Full-stack Development*, we are going to briefly cover various advanced full-stack development topics that have been left out of this book so far, giving you an idea of how to continue your journey of learning full-stack web development with React.